Basics of Harness & Loop engineering

By

Anand Kulkarni

anand.kulkarni1@zensar.com

Table of Content

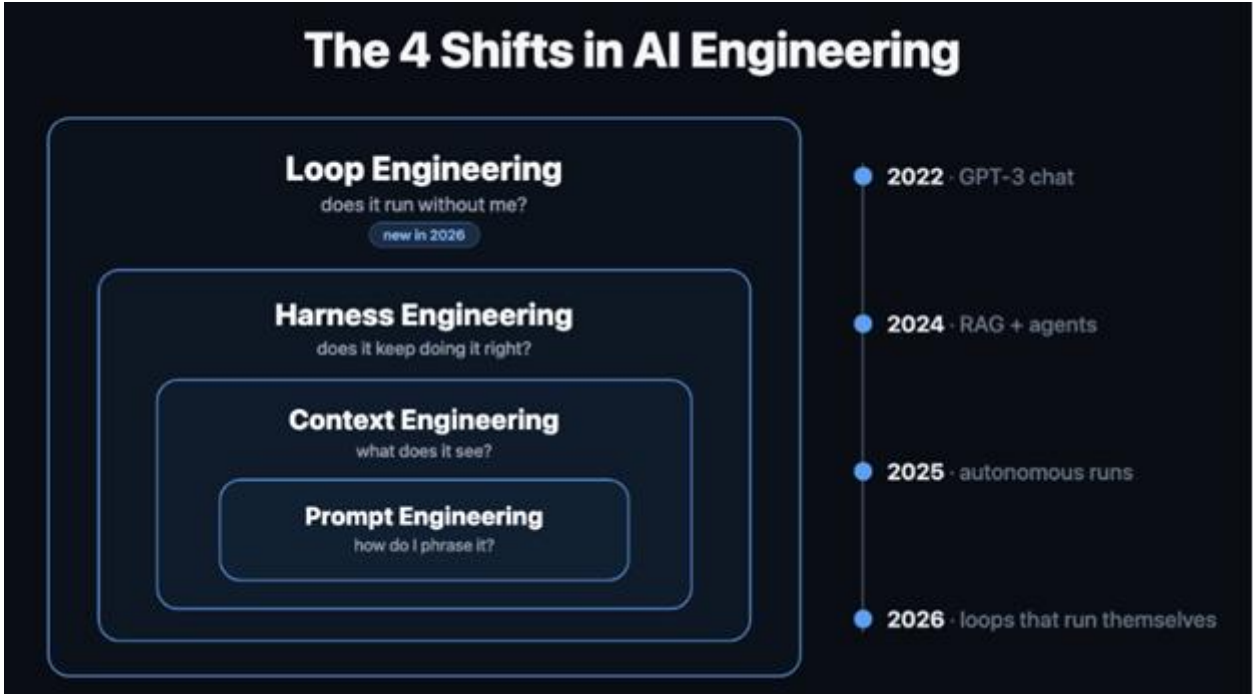
Module	Topic
Module 1:	Four shifts in AI engineering
Module 2:	Prompt engineering
Module 3:	Context engineering
Module 4:	Harness engineering
Module 5:	Loop engineering

Four shifts in AI engineering

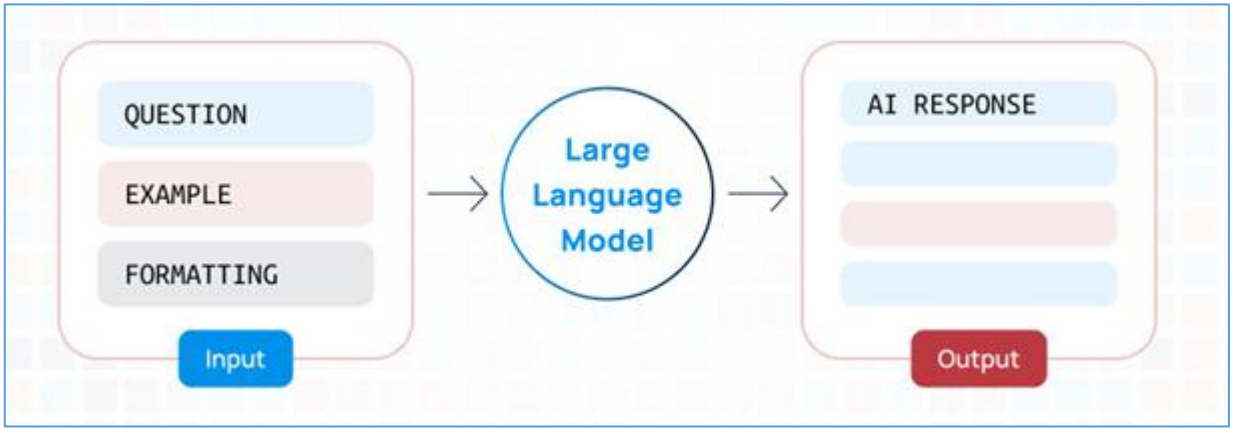
- In last few years, four terms are emerging in AI world: Prompt Engineering, Context Engineering, Harness Engineering, and now Loop Engineering.
- The reason behind these emerging terms is below questions raised for making LLM effective:

    1. Does the model understand what you're asking?
    2. Does the model have the right information?
    3. Does the model keep doing the right thing across a long, real execution?
    4. Does the work keep happening when you're not there to ask?

Four shifts in AI engineering

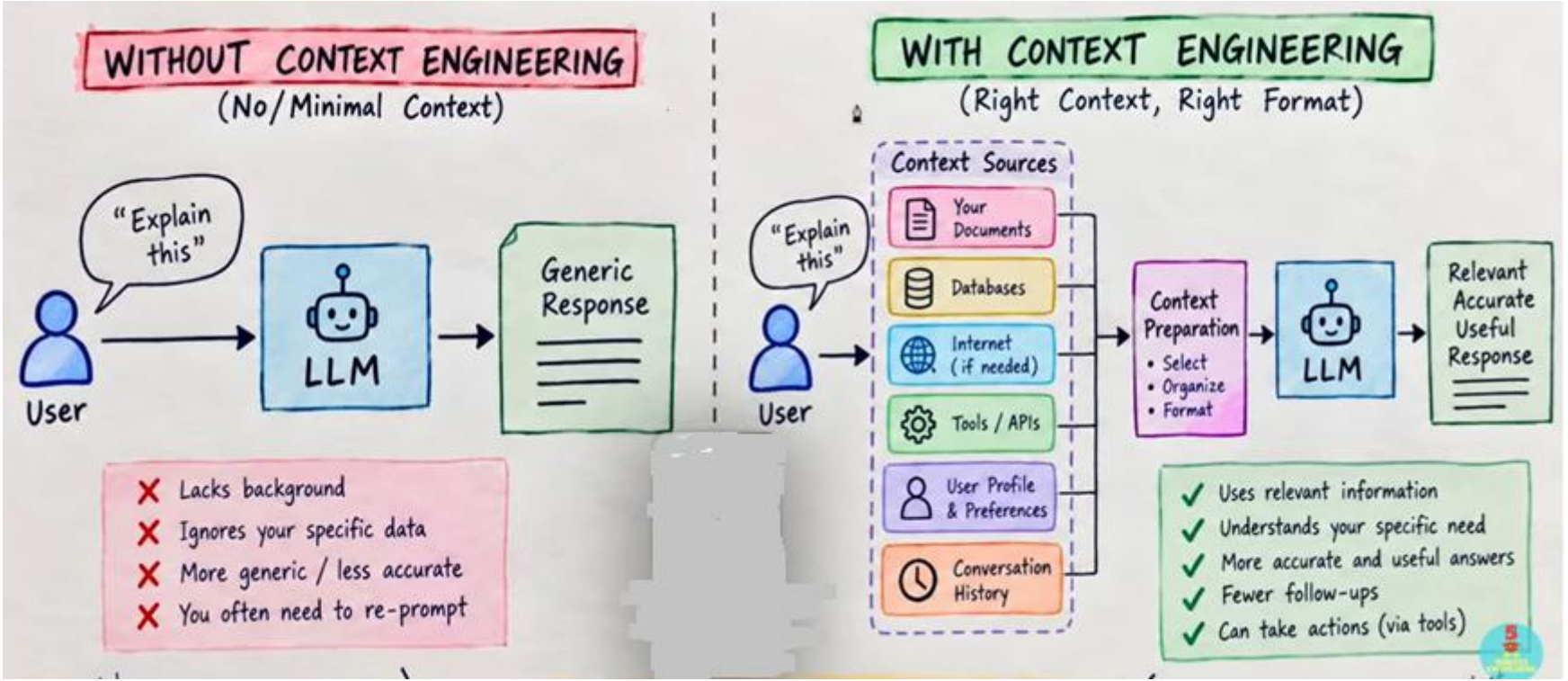


Shift 1: Prompt Engineering



- Prompt is an input or a query provided to a large language model (LLM)—to generate specific responses.
- Prompt can take various forms, such as question, command, or statement, and are designed to guide the AI in producing relevant and accurate outputs.
- The effectiveness of the response depends heavily on how well the prompt is crafted.

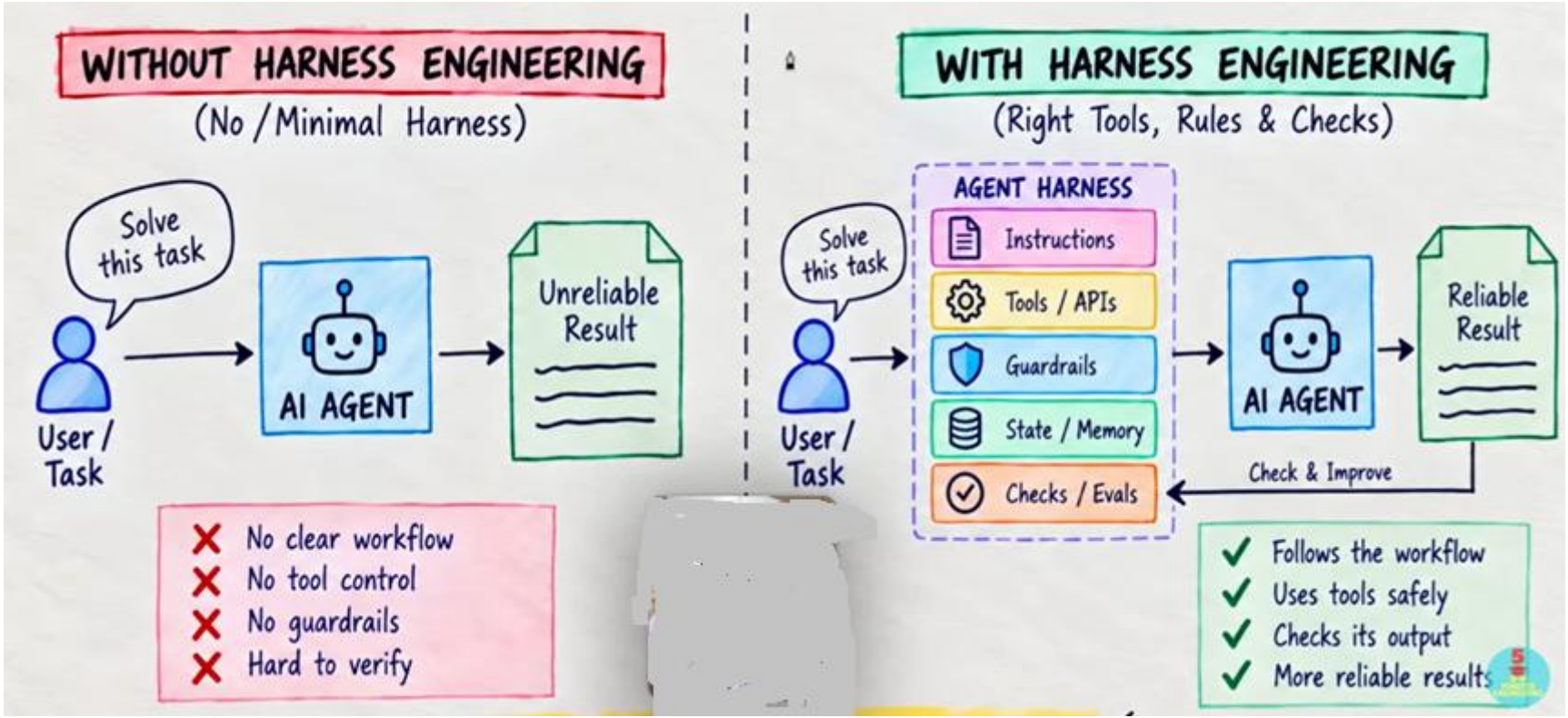
Shift 2: Context Engineering



Shift 2: Context Engineering

- Context engineering is the discipline of designing, structuring, and optimizing all information an AI system receives at inference time—far beyond prompt writing.
- Context engineering is emerging as the successor to prompt engineering, focusing on building complete, structured information environments that allow AI systems to reason effectively. It is now central to RAG systems, agentic AI, and enterprise AI reliability.

Shift 3: Harness engineering



Shift 3: Harness Engineering

- Harness engineering focuses on everything around the model — the runtime that couples an LLM to the real world.
- It defines how an agent perceives context, selects actions, executes tools, validates outputs, and maintains state across long tasks.
- Agent = Model + Harness
- Harness engineering solves problems that models alone cannot:
    1. Long-running task drift
    2. Missing or irrelevant context
    3. Tool-calling errors
    4. Uncontrolled retries
    5. Lack of verification or observability

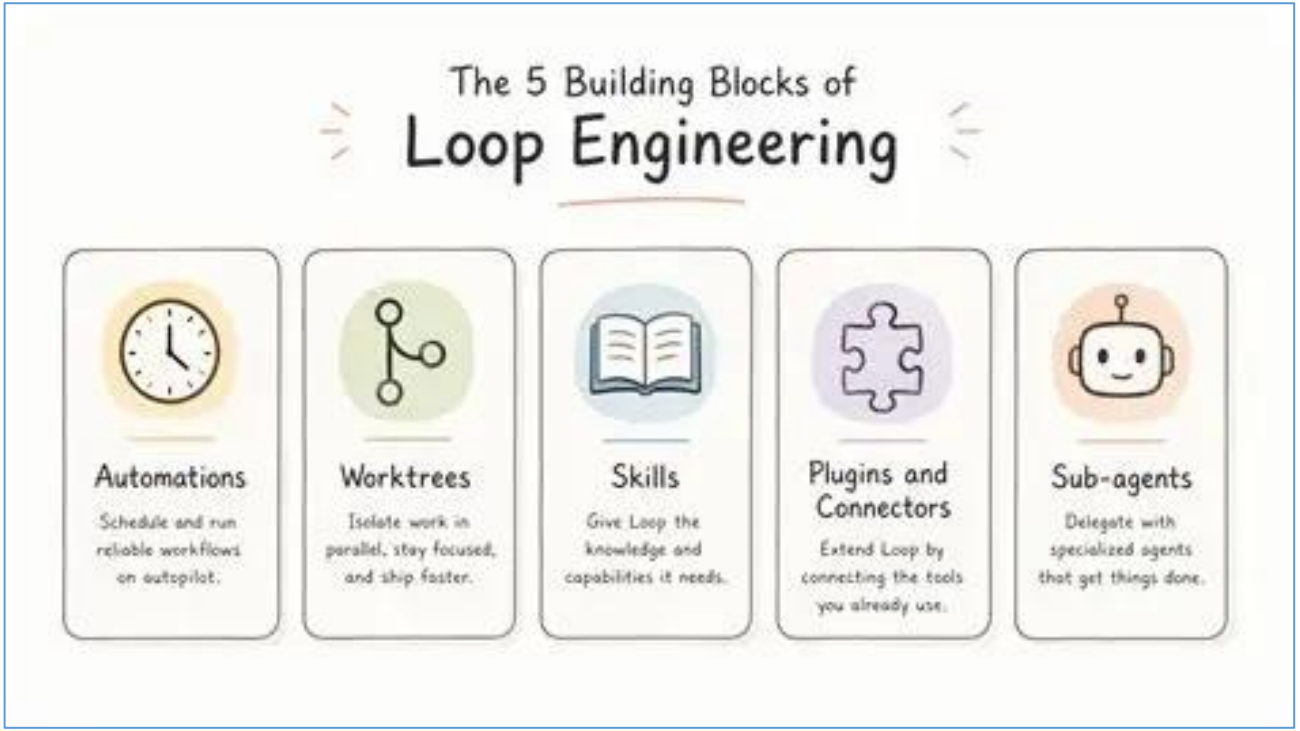
Components of Harness Engineering

- **Guides:** system prompts, AGENTS.md files, and constraint documents that direct what the agent knows and can do
- **Sensors:** evals, validation loops, output parsers, and drift detectors that observe and verify agent behavior
- **Data context layer:** the certified, lineage-verified data the harness supplies to the agent at runtime
- **Orchestration logic:** sequences agent tasks and routes outputs through review gates

Shift 4: Loop engineering

- Loop engineering is replacing yourself as the person who prompts the agent. You design the system that does it instead.
- A loop here can be thought of a recursive goal where you define a purpose and the AI iterates until complete.
- You shouldn’t be prompting coding agents anymore. You should be designing loops that prompt your agents.
- Similarly, Boris Cherny, head of Claude Code at Anthropic, said "I don’t prompt Claude anymore. I have loops running that prompt Claude and figuring out what to do. My job is to write loops".

Building blocks of Loop engineering



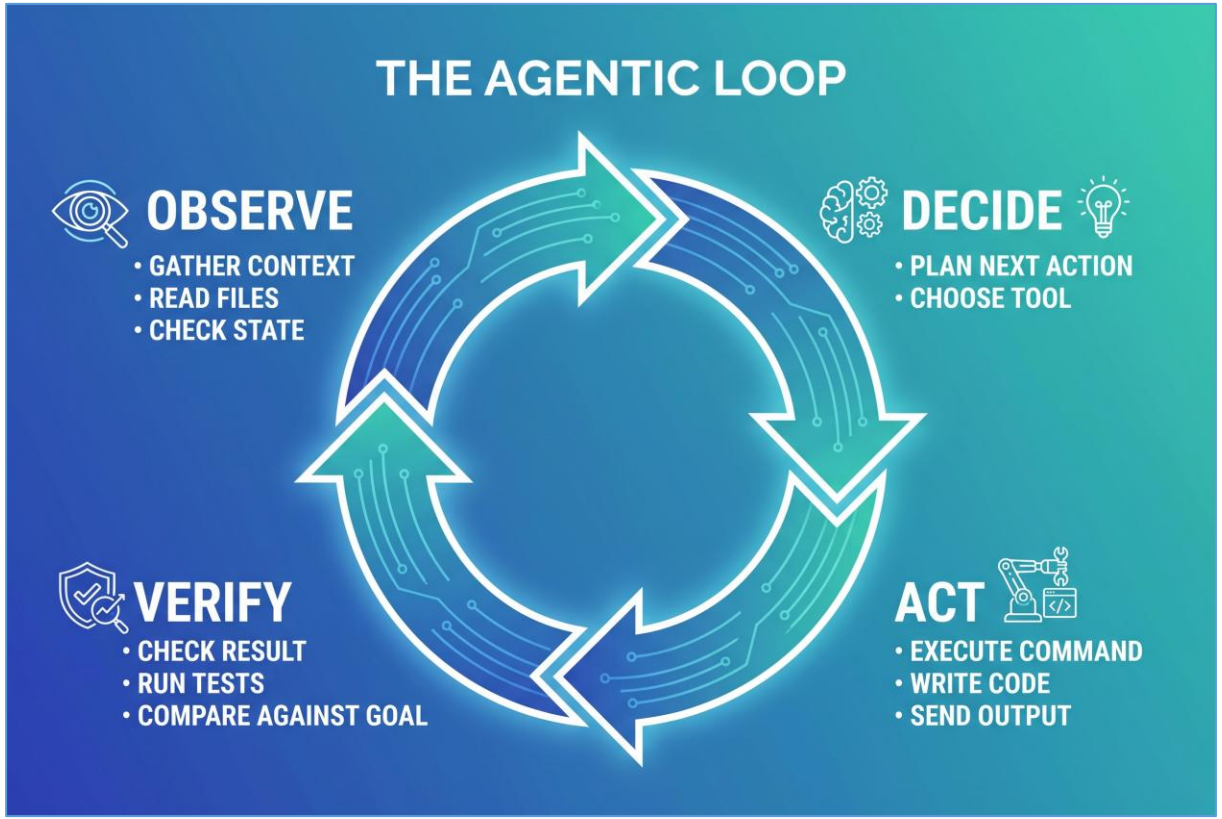
Building blocks of Loop engineering

Primitive	Job in the loop	Codex app	Claude Code
Automations	discovery + triage on a schedule	Automations tab: pick project, prompt, cadence, environment; results land in a Triage inbox; `/goal` for run-until-done	Scheduled tasks and cron, `/loop`, `/goal`, hooks, GitHub Actions
Worktrees	isolate parallel features	Built-in worktree per thread	`git worktree`, `--worktree`, `isolation: worktree` on a subagent
Skills	codify project knowledge	Agent Skills (`SKILL.md`), invoked with `$name` or implicitly	Agent Skills (`SKILL.md`)
Plugins / connectors	connect your tools	Connectors (MCP) plus plugins for distribution	MCP servers plus plugins
Sub-agents	ideate and verify	Subagents defined as TOML in `.codex/agents/`	Task subagents in `.claude/agents/`, agent teams
State	track what's done	Markdown or Linear via a connector	Markdown (`AGENTS.md`, progress files) or Linear via MCP

Building blocks of Loop engineering

- Automations that go off on a schedule and do discovery and triage by themselves.
- Worktrees so two agents working in parallel don’t step on each other.
- Skills to write down the project knowledge the agent would otherwise just guess.
- Plugins and connectors to plug the agent into the tools you already use.
- Sub-agents so one of them has the idea and a different one checks it.

Stages in Loop engineering



Thank you!!